

物联网系统综合课程设计报告撰写要求

(此页不装钉)

一、 纸张与页面要求

1. 采用国际标准 A4 型打印纸或复印纸，纵向打印。
2. 封页和页面按照下面模板书写（正文为：小四宋体 1.5 倍行距）。
3. 图表及图表标题按照模板中的表示书写。

二、 课设报告书的内容应包括以下各个部分：（按照以下顺序装订）

1. 封页
2. 成绩页
3. 任务书(学生教师均要签字，系主任签字/盖章,信息填写完整)
4. 课程设计总结
5. 目录

三、 正文一般应包括以下内容:

1. 概述：描述问题定义、可行性分析、项目实施计划（任务分解、系统开发分工、文档撰写分工、进度安排）。
2. 需求分析：描述题目背景、你对题目的理解、应该完成的功能、采用的工具等。
3. 概要设计：所设计的物联网系统的硬件整体结构、模块图(只用横线和竖线，不要有箭头)，图下用文字描述每个模块的基本功能；若有数据库设计，需画出数据库 E-R 图（图要表明实体之间的联系类型 1:1 或 1:n 等），对 E-R 图进行描述，说明实体之间为什么有联系，为什么是 1 对多等；数据库表属性概要说明；系统的功能模块层次图。
4. 详细设计：每个模块的具体实现过程需要进行详细说明，重要的模块要画出程序流程图；涉及到的数据库表结构详细说明，以表格的形式描述表结构（属性名称、属性类型、长度、属性含义），表格下方使用文字对数据库表进行说明（表的作用、字段的含义、如何使用等），该章节不需要粘贴系统的运行图和硬件的连线图。
5. 系统测试：可以图文并茂的描述各描述系统测试用例设计、测试方法、测试情况。
6. 参考文献：格式要求：[1]作者，等. 书名.出版地：出版社，出版年

沈阳航空航天大学

课 程 设 计 报 告

课程设计名称： **物联网系统综合课程设计**

课程设计题目： 指针型仪表的自动识别系
统

学 院： 人工智能学院

班 级： 物联网2202

姓 名： 陈梓欣

学 号： 223428010210

指导教师： 孙恩岩

完成时间： 2025 年 09 月 26 日

物联网系统综合课程设计报告

序号	评价项目	评分	
		满分	得分
1	课设态度，查阅资料和自我学习的能力。	10	
2	课题题目的理解情况和完成情况，以及仿真的正确性与完善性。	40	
3	检查过程中问题回答的准确程度	20	
4	课程设计报告的格式和内容，侧重考虑内容充实度、图表齐全度、对设计和实现过程的描述详实度、仿真和测试的全面度等方面。	30	
累计得分			
指导教师评语： _____ _____			
课程成绩：		指导教师签名 _____ 日期 年 月 日	

注：成绩评定采用五级记分制。优秀(90~100分)、良好(80~89分)、中等(70~79分)、及格(60~69分)、不及格(60分以下)

课程设计任务书

课程名称	物联网系统综合课程设计			专业	物联网工程		
学生姓名	陈梓欣		班级	物联网 2202		学号	223428010210
题目名称	指针型仪表的自动识别系统						
起止日期	2025	年	09	月	01	日起 至	2025 年 09 月 26 日止

课设内容和要求:

一、课程设计内容

利用配有摄像头的智能眼镜（或等效模拟设备）和 PC 机，设计并实现指针型仪表的自动识别系统。

二、课程设计要求

1. 利用智能眼镜摄像头采集指针型仪表图像数据；
2. 通过设备 Wi-Fi 模块实现图像数据传送功能；
3. 编写设备端程序，将图像数据发送到 PC 机；
4. 开发上位机程序，实现识别、数据存储与分析；

四、独立设计、调试和下载并通过指导教师现场验收；

五、撰写课程设计报告。

参考资料：

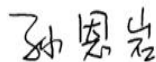
- [1]陈月，柴立平，朱利凯等. 基于仪表目标检测的光照自适应图像预处理方法研究[J]. 佳木斯大学学报(自然科学版), 2025, 43(03)
- [2]韩绍超,徐遵义,尹中川等. 指针式仪表自动读数识别技术的研究现状与发展[J]. 计算机科学, 2018, 45(S1):54-57.
- [3]李军锋,何双伯,冯伟夏等. 电力设备智能巡检中增强现实(AR)技术的应用研究[J]. 电工技术, 2018,(03):96-97.

4. 教研室审核意见：同意（√） 不同意（ ）

教研室主任签字：

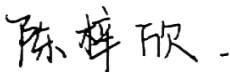


指导教师（签名）



2025 年 09 月 01 日

学生签名（签名）



2025 年 09 月 01 日

课程设计总结

本课程设计围绕“指针型仪表的自动识别系统”展开，目标是解决传统人工巡检效率低、易出错、缺乏实时性等问题。系统通过硬件摄像头采集仪表图像，并传输至上位机，上位机程序再完成识别、存储与分析，实现了从数据获取到结果展示的完整流程。

在设计过程中，我们首先进行了需求分析，明确了系统应具备图像采集与传输、仪表检测与校正、指针读数计算、数据存储以及用户管理等功能。随后完成了总体架构与详细模块设计：硬件端负责图像采集与传输，软件端则利用 PySide6 开发 GUI，实现用户交互；调用 YOLOv8 完成表盘检测，结合 OpenCV 进行透视校正和读数计算；通过 SQLite 数据库存储用户与结果信息，确保了系统的完整性与可扩展性。

系统测试表明，本设计功能运行稳定，读数平均误差小于 1%，响应时间控制在 2 秒以内，能够满足实时性与准确性要求。注册登录逻辑正确，GUI 操作简便直观，满足课程设计预期目标。

通过本次设计，我们掌握了从需求分析、系统开发到功能测试的完整流程，加深了对计算机视觉、数据库管理和软件工程方法的理解，并锻炼了团队分工与文档撰写能力。当然，系统仍有提升空间，如在复杂光照环境下的鲁棒性、数据分析与预测功能的扩展，以及算法向边缘端的移植。

总体而言，本项目不仅验证了基于智能眼镜的仪表自动识别方案的可行性与实用价值，也为后续科研与工程实践提供了经验与借鉴。

目 录

1	概述	1
1.1	问题定义	1
1.2	可行性分析	1
1.3	项目实施计划	2
2	需求分析	3
2.1	题目背景与理解	3
2.2	预计功能	3
2.3	所用工具	4
3	概要设计	5
3.1	物联网系统整体结构	5
3.1.1	硬件端模块功能说明	5
3.1.2	上位机端模块功能说明	6
3.2	数据库设计	7
3.2.1	E-R 图描述	8
3.2.1	数据库表属性概要说明	8
4	详细设计	10
4.1	用户注册与登录	10
4.2	表盘检测	12
4.2.1	表盘提取	12
4.2.2	表盘标注	13
4.3	图像校正与增强	14
4.3.1	表盘校正	15
4.3.2	图像增强	15
4.4	读数处理与结果显示	15

4.5 数据存储	17
5 系统测试	20
5.1 系统测试用例设计	20
5.2 测试方法	20
5.3 测试情况	21
5.3.1 用户注册与登录测试	21
5.3.2 图像采集与传输测试	23
5.3.3 表盘检测与读数计算测试	24
5.3.4 图像校正与增强测试	25
5.3.5 数据存储与回溯测试	26
参考文献	29
附录 关键部分程序清单	30

1 概述

1.1 问题定义

在工业生产和电力运行等关键场景中，大量指针型仪表仍被广泛用于监测压力、温度、电流、电压等重要参数。

传统的巡检模式主要依赖人工抄表，这种方式不仅效率低下，而且容易受到主观因素和环境条件的影响，导致误读、漏读等问题，进而影响运行安全和管理决策。此外，人工巡检缺乏实时性与数据记录能力，难以满足现代工业对于智能化与信息化的要求。例如，在电力运行中，如果关键设备的运行参数不能被实时准确采集，就可能延误故障的发现与处理，造成严重损失。

因此，亟需一种高效、智能、低成本的解决方案，能够替代人工实现仪表读数的自动识别与记录。本项目即基于此背景，提出并设计了一套指针型仪表的自动识别系统，以实现巡检工作的自动化、智能化和数据化管理，提升生产运行的安全性和管理效率。

1.2 可行性分析

从可行性角度看，本系统具备较强的实现基础。硬件方面，智能眼镜作为一种新型可穿戴设备，内置高分辨率摄像头，可实时采集现场仪表图像，同时其 Wi-Fi 模块可实现图像的无线传输，避免了传统数据线传输的不便。

上位机端采用性能良好的 PC 作为计算与分析平台，能够满足深度学习模型运行对算力的需求。

软件方面，系统采用 Python 作为主要编程语言，结合 PySide6 框架开发图形化用户界面，确保良好的人机交互体验；利用 OpenCV 进行图像预处理和透视校正，保证输入图像的质量；采用 Ultralytics YOLOv8 系列深度学习模型实现表盘检测和指针识别，准确率和鲁棒性均较高；数据库方面，选择轻量级的 SQLite 来实现用户信息与仪表读数的存储与管理，能够满足应用场景中对数据安全和查询便捷性的要求。

整体技术路径成熟清晰，均基于开源工具和框架，开发成本较低，且可扩展性强。因此，本系统在硬件与软件层面均具备落地条件，具有较高的可行性和应用价值。

1.3 项目实施计划

出于项目实施的周期考虑，为了保证进度与质量，项目计划划分为若干阶段，逐步推进。

首先，在第 1 周完成需求分析和总体设计工作，包括对题目背景的理解、系统功能需求的明确、技术路线的选择以及软件与硬件整体框架的设计。这一阶段重点在于打牢理论基础，保证后续开发有清晰的方向。

在第 2 周，主要开展硬件端与算法端的初步实现工作。硬件端重点是图像采集与数据传输，使其能够稳定地将图像传送到 PC；算法端则开始准备训练数据，对 YOLO 模型进行微调和测试，确保其能够对表盘和指针区域进行有效检测。

第 3 周的工作集中在软件端开发与功能整合。主要任务包括基于 PySide6 实现 GUI 界面，完成用户注册与登录功能，调用 OpenCV 实现图像预处理与校正，并结合 SQLite 数据库进行数据存储与管理。同时，还需编写接口函数，将检测结果与界面联动展示。

最后，在第 4 周进入系统集成与测试阶段。将前端数据采集、图像检测、读数计算和数据存储模块进行整合，完成整体功能调试，进行误差分析与性能优化，并整理实验数据与结果，完成课程设计文档的撰写与总结，确保项目在周期内完整实现。

2 需求分析

2.1 题目背景与理解

在工业现场和电力运行系统中，指针型仪表依然具有不可替代的作用。虽然数字仪表与智能传感器逐渐普及，但由于指针型仪表具有结构简单、稳定性高、抗干扰性强和成本低廉等优点，仍广泛应用于电力、化工、冶金、交通运输及实验室设备中。

例如，在高温、高湿或强电磁环境下，数字仪表往往易受干扰，而机械式指针表则能保持可靠运行。然而，长期以来，人工巡检和人工抄录一直是最主要的读数方式。

本项目的核心任务是设计指针型仪表的自动识别系统。该系统不仅是一个简单的图像识别工具，更是一个软硬件一体化的综合应用，涵盖了图像采集、无线通信、机器视觉、数据库管理以及用户交互界面等多个技术环节。系统需要在保证识别精度的同时兼顾实用性，以适应现场巡检的需求；还要具备用户管理与数据存储功能，实现对历史读数的查询与分析。通过这种方式，传统的人工巡检模式可以被替代，从而实现自动化、智能化的数据采集和管理，具有较强的应用推广价值。

2.2 预计功能

硬件端功能：由硬件设备进行图像采集与数据传输，保证上位机能够实时接收仪表图像。

图像接收模块：PC 端需要具备稳定接收数据的能力。接收后将图像存入指定路径，同时完成数据格式转换和缓存处理，确保后续识别任务可以直接调用，避免丢包或传输延迟带来的影响。

仪表识别模块：基于 YOLOv8 的深度学习模型，实现表盘区域检测和指针定位。该模块需具备较强的鲁棒性，能在光照变化和角度偏移的条件下保持准确度，并为后续透视校正和读数计算提供基础。

图像校正与增强模块：对检测出的仪表图像进行透视校正，消除拍摄角度带来的畸变；在获取经过校正后的仪表图像后，进一步对其进行图像增强处理，使其更加便于后续识别。

仪表读数模块：结合刻度特征与指针角度计算出具体读数。该模块要求在保证精度的同时，运行速度足够快，满足实时巡检需求。

数据存储模块：通过 SQLite 数据库存储用户信息与读数数据。该模块不仅要保证密码安全性（如哈希存储），还要实现数据的长期保存、快速检索和后续统计分析，形成可靠的数据支撑。

上位机 GUI：通过 PySide6 实现可视化界面，支持用户注册、登录、图像显示和识别结果查看。界面需简洁明了，操作直观，保证用户能够轻松完成导入、拍摄和查看结果等操作。

2.3 所用工具

硬件：智能眼镜作为前端数据采集与传输设备，配合 PC 机作为上位机平台。PC 具备较高的运算性能，能够支持深度学习模型的推理与结果可视化。

编程语言：Python 3.x，具有丰富的开源生态与跨平台优势，能够快速开发图像处理、模型调用和界面交互功能，是本系统的核心语言。

深度学习框架：PyTorch 结合 Ultralytics YOLOv8 系列，用于仪表检测与指针识别。YOLOv8 系列深度学习模型具备较高的检测速度与精度，适合工业应用的实时性要求。

图像处理工具：OpenCV，用于透视校正、图像裁剪与边缘检测。其函数库成熟稳定，能够为后续角度计算与读数解析提供可靠支持。

数据库：SQLite，轻量级、免配置，适合桌面级应用的数据存储需求。既能保存用户信息与登录状态，也能记录仪表读数及时间戳。

界面开发工具：PySide6，支持跨平台 GUI 开发。可用于实现多页面切换、图像显示、按钮功能绑定，保证良好的人机交互体验。

3 概要设计

3.1 物联网系统整体结构

本项目涉及到的系统整体结构模块图如图 3.1 所示。

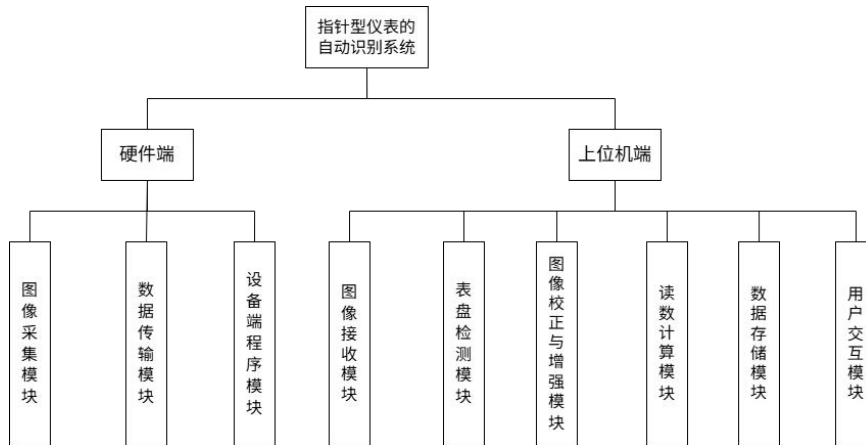


图 3.1 系统整体模块图

3.1.1 硬件端模块功能说明

1) 图像采集模块

该模块利用高清摄像头对仪表进行实时拍摄。其核心目标是保证图像的清晰度和稳定性，避免因抖动或模糊影响后续的识别。采集的图像需覆盖整个表盘，并保证光照条件下指针和刻度能够清晰区分。

2) 数据传输模块

该模块负责将拍摄的图像数据传输至上位机。其核心是确保数据传输过程的稳定性和完整性，提升系统的便捷性和实时性。

3) 设备端程序模块

该模块相当于硬件端的调度与控制核心，负责调用摄像头和传输模块，管理图像的采集与发送。它能自动建立与上位机的通信连接，并将采集到的图像打包传输，保证数据按顺序到达上位机，避免丢包。

3.1.2 上位机端模块功能说明

1) 图像接收模块

该模块主要任务是接收来自智能眼镜端传输的图像数据，并将其保存到本地缓存目录。接收到的图像需要进行基础处理（如格式转换和路径管理），为后续识别与分析提供数据输入保障。

2) 表盘检测模块

借助深度学习框架 YOLOv8，实现对仪表表盘和指针的自动检测。该模块输出检测框信息，包括表盘的边界和指针的位置坐标，并过滤掉低置信度的检测结果。其准确性直接决定系统的整体性能，是核心环节之一。

3) 图像校正与增强模块

针对现场拍摄存在的角度偏差和透视畸变问题，该模块利用 OpenCV 实现透视变换，将倾斜的表盘图像校正为标准的正视图；在获取经过校正后的仪表图像后，进一步通过亮度对比度调整（alpha, beta）、Gamma 校正、CLAHE 局部对比度增强等方法。处理后的图像可大幅提升读数计算的准确度，保证后续刻度和指针识别的一致性。

4) 读数计算模块

根据校正后的表盘和检测到的指针位置，利用几何计算方法（如 atan2 算法）求得指针与基准刻度的角度。结合仪表的刻度范围和比例，计算出最终读数。该模块要求在精度与速度之间保持平衡，确保实时性和可靠性。

5) 数据存储模块

该模块基于 SQLite 数据库实现数据持久化管理。它保存用户账户信息（用户名、加密后的密码）、仪表读数以及采集时间。数据库支持历史数据查询与导出，为后期数据分析和可追溯性提供支撑。

6) 用户交互模块

首先，它提供注册与登录功能，所有密码均经过哈希加密后存储，确保用户信息安全。系统在登录时会验证输入信息，只有合法用户才能进入操作界面，避免未授权访问。同时，基于 PySide6 的图形界面为用户提供友好的操作平台，包含图像显示区、功能按钮区与结果展示区。用户既可以手动导入图像，也可调用摄像头实时采集，并执行校正与识别操作。识别结果直观显示。

3.2 数据库设计

为了满足系统在用户管理和数据存储方面的需求，本系统设计了五个核心实体：用户表（User），任务表（Task）和图片路径存储表（Image_record）、矩形框坐标存储表（Rectangle）与旋转框坐标存储表（Rotate）。

这种数据库结构设计既保证了数据的完整性和一致性，又简化了查询操作，能够高效支持用户登录验证、历史结果查询和统计分析等功能，满足本系统的应用需求，所用数据库 E-R 图如图 3.2 所示。

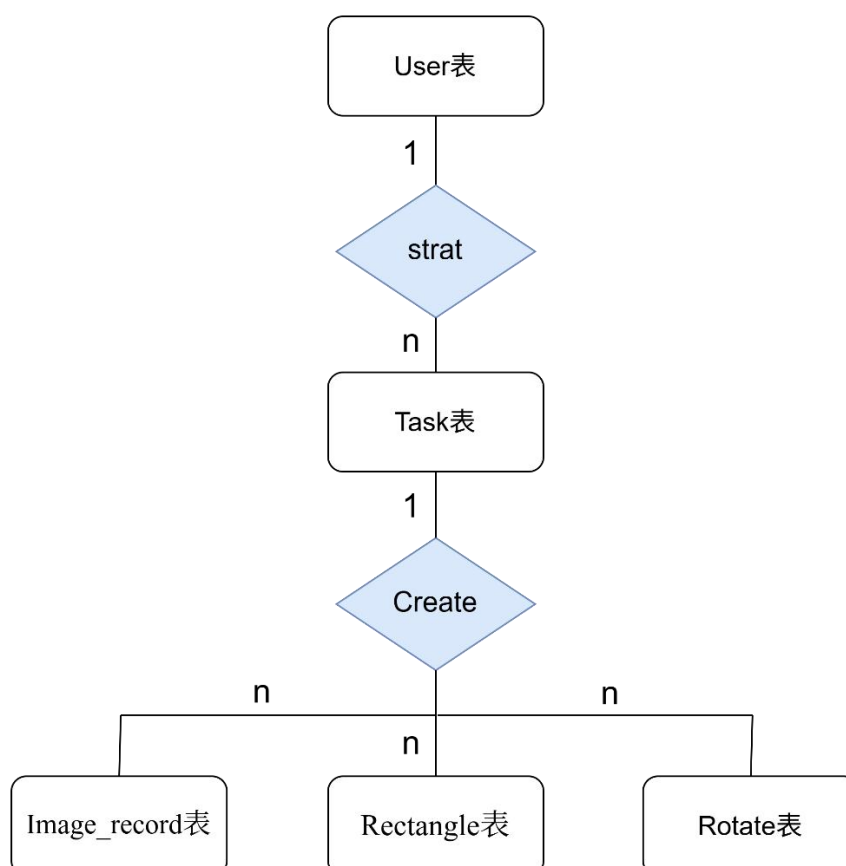


图 3.2 数据库 E-R 图

3.2.1 E-R 图描述

1) user $\rightarrow$ task (1:n)

每个用户可发起多次识别任务,一个任务只能属于一个用户,因此是 一对多。

通过 task.user_id 外键关联 user.user_id, 支持按用户统计任务。

2) task $\rightarrow$ image_record、rectangle 与 rotate (1:n)

每次识别任务可能生成多个阶段的图片记录(原图、分割图、校正增强图、YOLO/OBB 结果等), 一张图片只能属于一个任务, 因此是 一对多。

通过 image_record.task_id 外键关联 task.task_id, 便于任务追溯和结果可视化。

矩形框和旋转框分别独立存储框坐标信息, 便于图像处理或模型训练。框表中未直接与任务或图片关联, 可在业务逻辑中按任务/图片 ID 分组使用。

3.2.1 数据库表属性概要说明

- 1) 用户表 (user): 用于存储系统用户信息, 其中 user_id 为主键, 自增, 唯一标识用户; username 和 password 为必填字段, 分别存储用户名和密码; created_at 记录用户注册时间。
- 2) 任务表 (task): 用于记录每一次识别任务, 其中 task_id 为主键, 自增, 唯一标识任务; user_id 为外键, 关联 user 表, 用于标识任务所属用户; timestamp 记录任务创建时间; result_value 用于保存最终的数值结果。
- 3) 图片记录表 (image_record): 用于保存任务在各阶段生成的图片信息, 其中 image_id 为主键, 自增; task_id 为外键, 关联 task 表, 指明图片所属任务; stage 字段为图片所属阶段; image_path 保存图片文件路径; created_at 记录图片记录的创建时间。
- 4) 矩形框表 (rectangle): 用于存储矩形标注框信息, 其中 id 为主键, 自增; classes_id 表示框所属类别; cx、cy 为框中心坐标; x1, y1 ... x4, y4 为矩形四个顶点坐标, 用于精确描述矩形形状。
- 5) 旋转框表 (rotate): 用于存储旋转标注框信息, 表结构与矩形框表结构

基本相同。不同点为 $x_1, y_1 \dots x_4, y_4$ 为旋转框四个顶点坐标，用于描述倾斜或旋转的目标框。

4 详细设计

4.1 用户注册与登录

该模块基于 PyQt 提供图形界面，实现用户注册与登录功能。系统启动时，通过 `init_db()` 方法初始化 SQLite 数据库 `user.db`，若 `users` 表不存在则自动创建，表中包含 `user_id`（用户 ID）、`username`（用户名）、`password`（哈希密码）和 `created_at`（用户创建时间）字段。

涉及到的数据库结构如表 4.1 所示，用户注册与登录具体实现流程如图 4.1 所示。

注册功能中，首先获取用户输入的用户名、密码及确认密码，检查密码一致性和非空性，再调用 `hash_password()` 进行哈希处理，然后将信息插入数据库。若用户名已存在，则捕获 `sqlite3.IntegrityError` 并弹窗提示。

登录功能中，获取输入的用户名和密码，哈希密码后查询数据库匹配，若用户名不存在或密码错误，均通过弹窗提示用户，登录成功后进入操作界面。

整个模块通过 `QMessageBox` 弹窗提供交互反馈，确保用户输入有效性及安全性。该设计同时保证密码不以明文形式存储，增强系统安全性。

表 4.1 用户表结构

表名	属性名称	属性类型	属性含义
user	user_id	INTEGER PK	用户唯一 ID，自增
	password	TEXT NOT NULL	用户名
	password	TEXT NOT NULL	密码（哈希存储）
	created_at	TIMESTAMP	用户创建时间

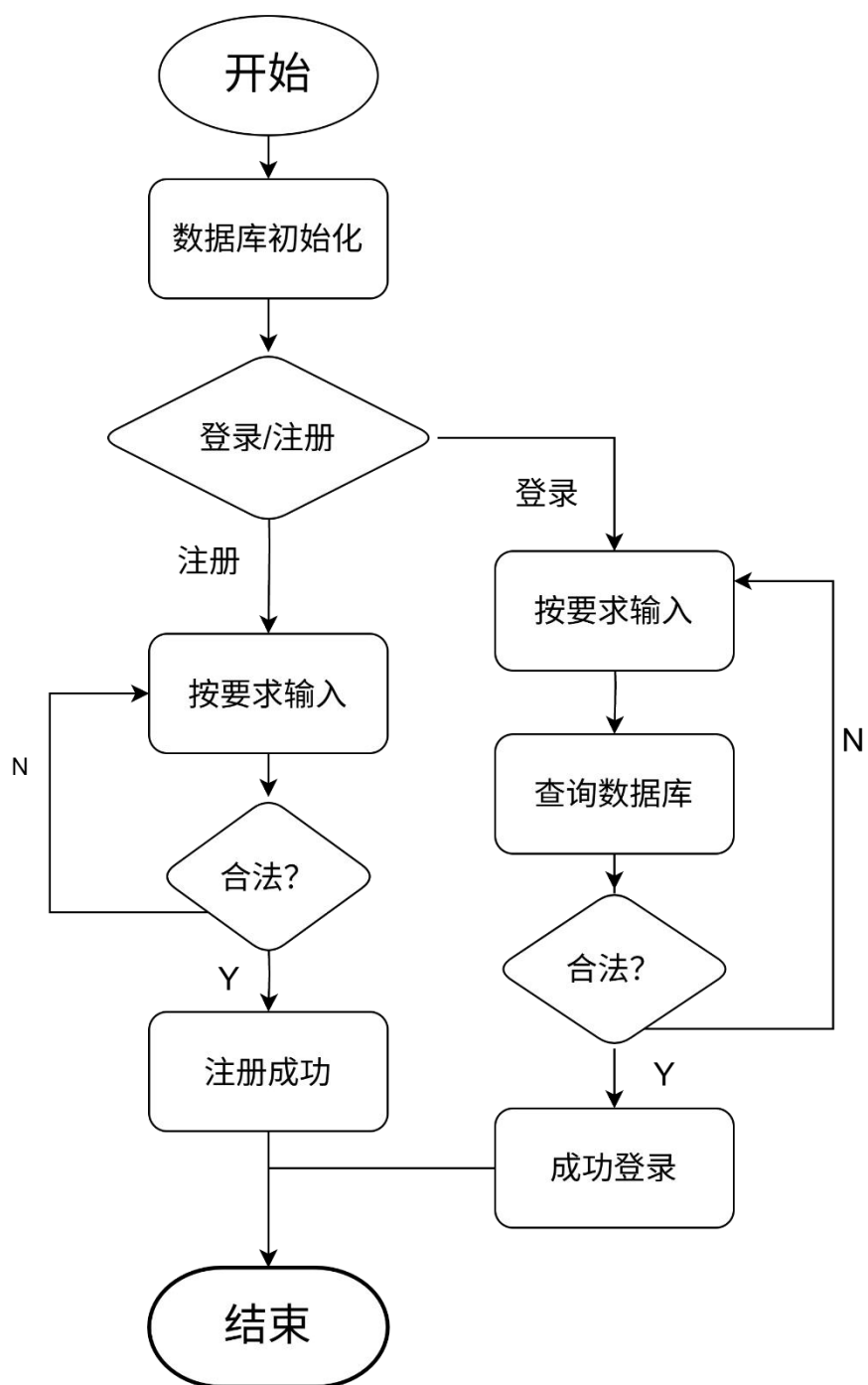


图 4.1 用户注册与登录流程图

4.2 表盘检测

4.2.1 表盘提取

程序首先检查输入图片路径是否存在，如果不存在直接返回错误。若路径有效，则加载指定的 YOLO 目标检测模型，并读取输入图片进行推理。模型输出的检测结果中包含多个候选框，程序会根据置信度阈值（ ≥ 0.7 ）过滤掉低置信度目标。

若存在符合条件的目标框，则标记为“检测到目标”。随后，程序对每个目标框的坐标进行解析。若未检测到任何表盘，则提示“输入图片非法”。具体实现流程如图 4.2 所示。

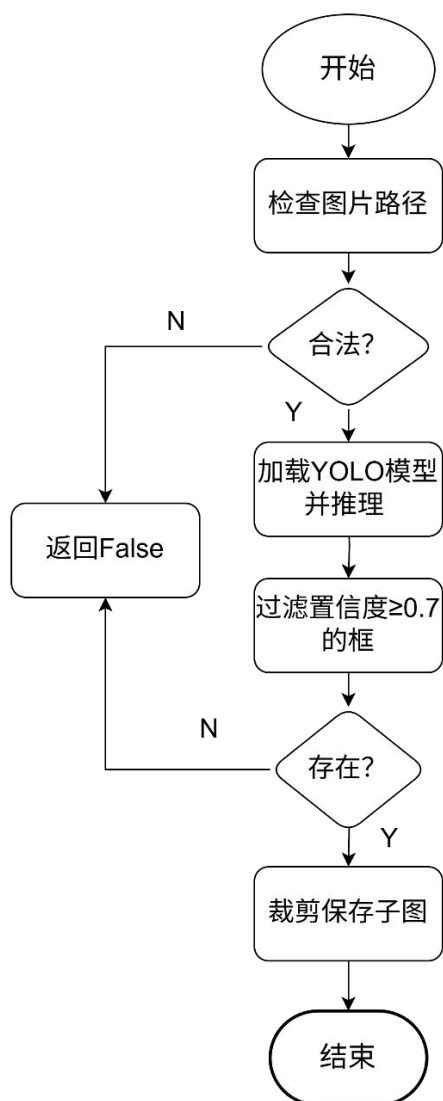


图 4.2 表盘提取流程图

4.2.2 表盘标注

该功能的主要作用是对输入的表盘图像进行检测与标注，并将检测结果结构化保存。程序首先加载训练好的 OBB（定向边界框）模型，对表盘图像进行推理，得到候选目标框。随后，系统会过滤掉置信度低于 0.7 的目标，只保留可靠的检测结果。

对于每个检测框，程序提取其中心坐标、四个顶点坐标及对应类别标签，并将这些信息统一存储在列表中。为了保证输出的逻辑性，检测框会按照横坐标进行排序。接着，程序将结果格式化为文本内容，写入到数据库，实现检测记录的累积追踪。

最后，程序还会生成可视化图片，将检测到的目标框绘制并保存。整体逻辑实现了表盘刻度与指针等元素的自动标注与记录，既便于后续分析，又保证了历史数据可追溯。若检测到非指针型仪表，则提示“输入仪表类型非法”。

具体实现流程如图 4.3 所示。

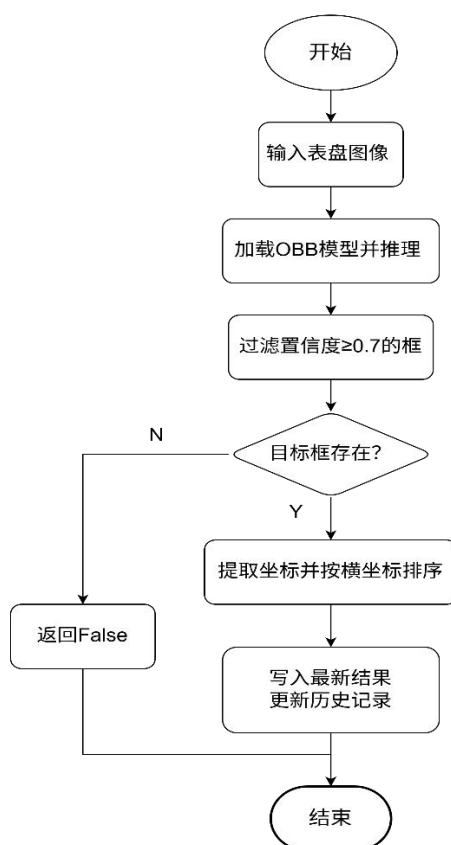


图 4.3 表盘标注流程图

4.3 图像校正与增强

该功能的主要目标是对指针型仪表图像进行自动表盘裁剪、透视校正，并结合多重图像增强方法生成带透明背景的高质量 PNG 图片，具体实现流程如图 4.4 所示。

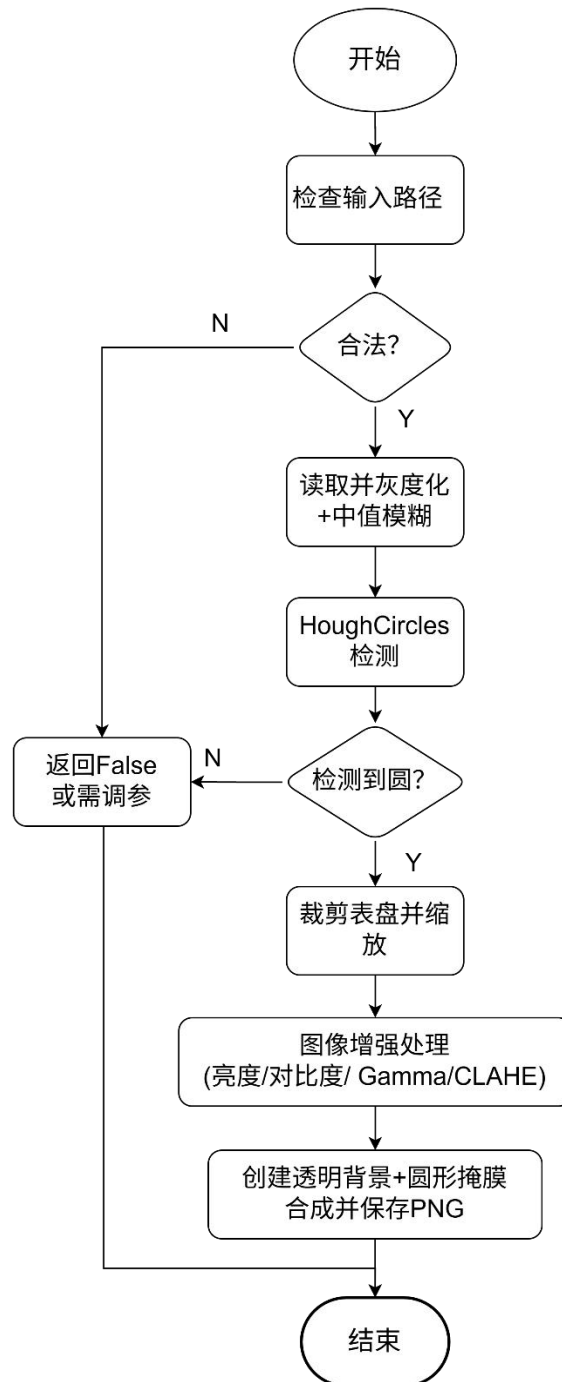


图 4.4 图像校正与增强流程图

4.3.1 表盘校正

程序首先检查输入路径是否存在并成功读取图像，否则直接报错并退出。读取的图像会先转为灰度图并进行中值模糊，以降低噪声，便于后续圆形检测。

随后利用霍夫圆变换（HoughCircles）检测表盘外圆，若未检测到圆则提示调整参数；若检测成功，则提取圆心与半径，并在原图中裁剪出表盘区域。为保证表盘完整，半径会适当放大，裁剪后的表盘图像再缩放为指定大小的正方形。

4.3.2 图像增强

接着进行图像增强：先通过线性变换调整对比度和亮度，再进行 Gamma 校正改善灰度层次，同时在 LAB 色彩空间对亮度通道应用 CLAHE（自适应直方图均衡化），以增强局部对比度。增强后的图像会被叠加到一个带有透明通道的 RGBA 图层上，透明通道通过圆形掩膜生成，确保背景透明，仅保留表盘内容。最后保存为 PNG 文件，并返回处理结果。该流程保证了表盘图像的清晰度与独立性，便于后续检测与分析。

4.4 读数处理与结果显示

该功能的核心目标是从检测结果文件中提取刻度与指针位置信息，经过几何计算得到仪表的读数，并将结果保存便于追溯。具体实现流程如图 4.5 所示。

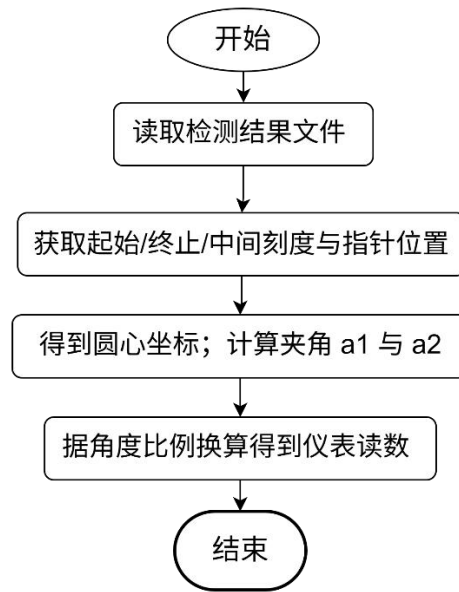


图 4.5 读数处理与结果显示流程图

程序首先通过 `read_scale_latest_file` 函数读取最新的检测文本文件，并利用正则表达式提取每个检测框的标签、中心点坐标及四个角点信息，形成结构化数据。

接着在 `compute_gauge` 中定位关键元素：起始刻度、终止刻度、中间刻度以及指针端点。

随后，程序通过几何方法计算仪表圆心：分别求三组点的垂直平分线，再求交点，最终取平均作为圆心坐标。

已知表盘圆心坐标为 (x_c, y_c) ，指针指向点坐标为 (x_p, y_p) ，可计算指针与水平方向的夹角 θ ：

$$\theta = \arctan\left(\frac{y_p - y_c}{x_p - x_c}\right)$$

如果需要换算成 0° 到 360° 的角度，则需进一步调整：

$$\theta' = \begin{cases} \theta + 180^\circ, & x_p < x_c \\ \theta, & x_p \geq x_c \end{cases}$$

若存在坐标偏差则进行修正。以圆心为基准，构造起始刻度、终止刻度与指针的向量，并计算其夹角。

若仪表的最小刻度为 $R_{\min}$ ，最大刻度为 $R_{\max}$ ，指针角度范围为 $\theta_{\min}$ 到 $\theta_{\max}$ ，则最终的读数 R 的计算如下：

$$R = R_{min} + \frac{(\theta' - \theta_{min}) \times (R_{max} - R_{min})}{\theta_{max} - \theta_{min}}$$

如果读数超过了仪表的最大可读范围，即 $R > R_{max}$ ，系统能够自动进行校正并提醒用户。

通过夹角比例，结合刻度范围，推算出最终的仪表读数。

若参考刻度点坐标为 (x_r, y_r) ，则指针与参考刻度之间的夹角 α 计算如下：

$$\alpha = \arccos \left(\frac{(x_p - x_c)(x_r - x_c) + (y_p - y_c)(y_r - y_c)}{\sqrt{(x_p - x_c)^2 + (y_p - y_c)^2} \times \sqrt{(x_r - x_c)^2 + (y_r - y_c)^2}} \right)$$

然后，利用角度与仪表范围之间的关系，通过公式计算出最终的读数。

$$R = R_{min} + \frac{\alpha \times (R_{max} - R_{min})}{\theta_{max} - \theta_{min}}$$

在此过程中，系统还会对角度值进行判定，确保其在合理范围内，若出现异常（如角度超出 360° 或负值情况），则进行错误提示。

所有计算过程和中间结果（包括圆心坐标、夹角大小和最终数值）会被格式化输出，并写入日志文件。日志采用时间戳记录，从而实现对仪表检测的可视化与可溯源化管理。

4.5 数据存储

本数据存储模块基于 SQLite 实现，用于管理用户信息、识别任务及各阶段的图片记录，同时支持矩形框和旋转框的存储。模块初始化时，首先建立数据库连接，并依次创建表，涉及到的数据库结构如表 4.2 所示，具体实现流程如图 4.6 所示。

其中，`user` 表存储用户基本信息，包括用户名、密码及创建时间；`task` 表记录每一次识别任务，关联对应用户，同时保存任务生成的最终读数；`image_record` 表记录任务中每个阶段的图片路径；`rectangle` 和 `rotate` 表分别用于存储矩形框和旋转框坐标信息。

模块提供插入方法，包括 `insert_task`、`insert_image`、`insert_rectangle` 和 `insert_rotate`，可在识别流程中动态保存数据，通过 `commit()`，保证数据持久化。

模块最后提供 `close()` 方法安全关闭数据库连接，防止资源泄露。整个模块逻辑清晰，支持按用户和任务追踪每一次识别操作及对应图像及框信息，实现了

完整的数据管理功能。

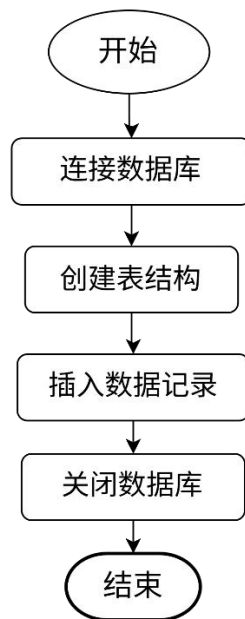


图 4.5 数据存储流程图

表 4.2 数据存储表结构

表名	属性名称	属性类型	说明
task	task_id	INTEGER PK	任务唯一 ID, 自增
	user_id	INTEGER FK	关联用户 ID
	timestamp	TIMESTAMP	任务创建时间
	result_value	REAL	最终识别结果
image_record	image_id	INTEGER PK	图片记录唯一 ID
	task_id	INTEGER FK	关联任务 ID
	stage	TEXT	图片处理阶段
	image_path	TEXT	图片路径
	created_at	TIMESTAMP	图片记录创建时间
rectangle	id	INTEGER PK	矩形框 ID, 自增
	classes_id	INTEGER	框类别 ID
	cx, cy	INTEGER	框中心坐标
	x1, y1... x4, y4	INTEGER	四个顶点坐标
rotate	id	INTEGER PK	旋转框 ID, 自增
	classes_id	INTEGER	框类别 ID
	cx, cy	INTEGER	框中心坐标
	x1, y1... x4, y4	INTEGER	四个顶点坐标

5 系统测试

5.1 系统测试用例设计

本项目在进行系统测试阶段所使用的全系统测试用例如表 5.1 所示。

表 5.1 系统测试用例

测试编号	测试模块	测试内容	输入条件	预期结果
TC-01	用户注册/登录	注册新用户并登录	新用户名、新密码	用户注册成功并能登录系统，数据库生成新记录
TC-02	图像采集与传输	硬件采集仪表图像以及数据传输	模拟摄像头采集图像	上位机接收图像完整、无丢帧
TC-03	表盘检测	YOLOv8 检测表盘区域及指针位置	校正后的仪表图像	返回正确的边界框和指针坐标
TC-04	图像校正与增强	对倾斜/模糊图像进行透视校正与增强	各类角度拍摄的仪表图片	图像校正后为正视图，增强清晰可读
TC-05	读数计算	根据检测结果计算仪表读数	指针位置及刻度信息	读数精度误差 $\leq 1\%$ ，结果输出至日志文件
TC-06	数据存储	保存任务及图片信息至 SQLite	多次识别任务及图片	数据库正确插入对应记录，可查询历史记录
TC-07	GUI 操作	导入图片、实时采集、识别操作	用户操作界面	操作流程无报错，结果可视化显示正确
TC-08	异常处理	输入非法图片、重复注册	非指针型图片、已存在用户名	系统弹窗提示错误，程序稳定运行

5.2 测试方法

1) 功能测试

按照系统需求文档逐一执行每个功能模块，验证用户注册登录、图像采集、表盘检测、图像校正、读数计算及数据存储等功能是否满足预期。测试重点在于功能完整性和正确性，确保各模块能协同工作，实现端到端的自动识别流程。

2) 黑盒测试

对系统输入和输出进行验证，通过导入不同类型的指针型仪表图像，测试表盘与指针检测准确性，以及图像校正和读数计算是否符合预期，确保系统在多场景下均能稳定运行。

3) 边界值测试

对输入图像尺寸、指针角度、置信度阈值等关键参数进行极限测试，验证系统在边界条件下的鲁棒性。通过观察系统响应和输出结果，评估算法是否能在非理想条件下仍保持识别准确性与程序稳定性。

4) 性能测试

测试图像从采集、传输到识别、显示的整体响应时间，并评估批量任务处理效率。通过记录各环节耗时，确保系统在实时性要求下稳定运行。

5) 异常处理测试

输入非法图像、重复注册等异常情况，验证系统是否能正确提示用户并保持稳定。测试重点是系统的容错能力和稳定性，确保在异常操作程序仍可稳定运行。

5.3 测试情况

5.3.1 用户注册与登录测试

通过新用户注册、重复用户名、密码错误等场景测试，系统均能正确响应。注册成功后用户信息准确保存至数据库，登录验证机制正常工作，操作界面提示明确，用户体验流畅且安全性得到保证。程序初始化效果如图 5.1 所示、新用户注册相关测试结果如图 5.2 所示、重复注册与新用户未注册相关测试结果如图 5.3 所示、用户（密码）为空与登录密码错误相关测试结果如图 5.4 所示。



图 5.1 程序初始化

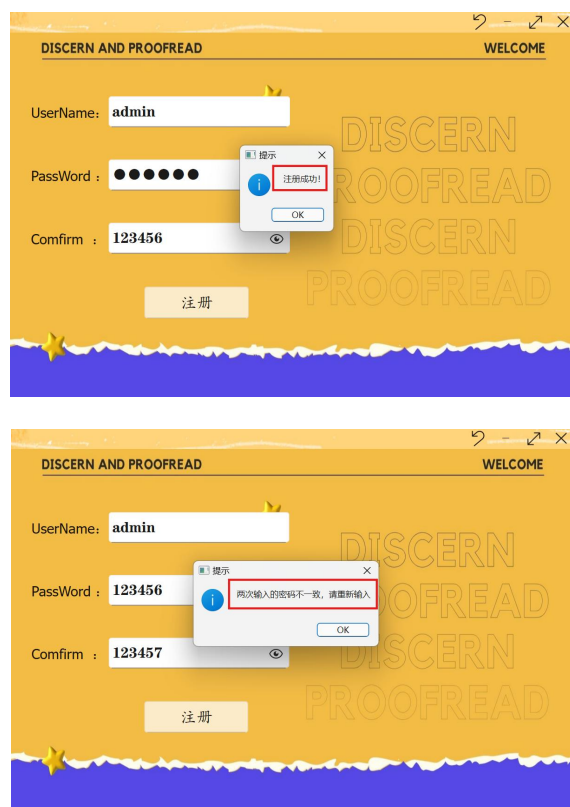


图 5.2 新用户注册

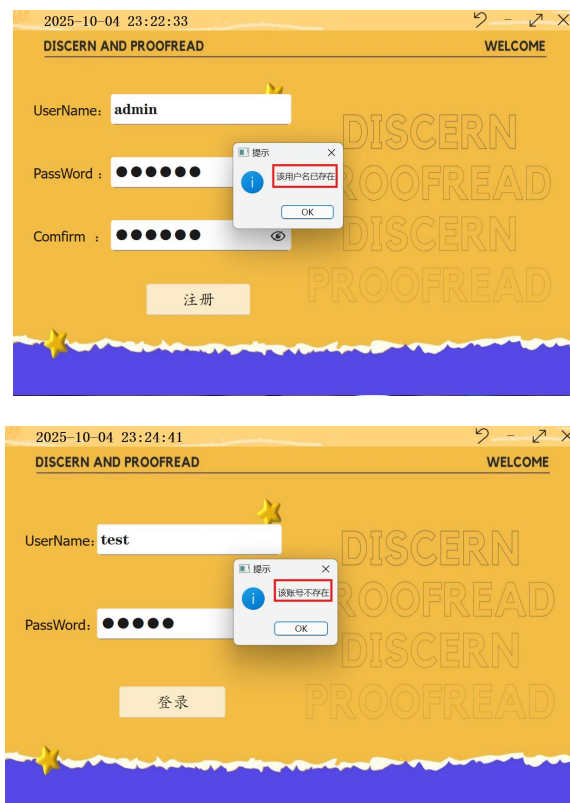


图 5.3 重复注册与新用户未注册

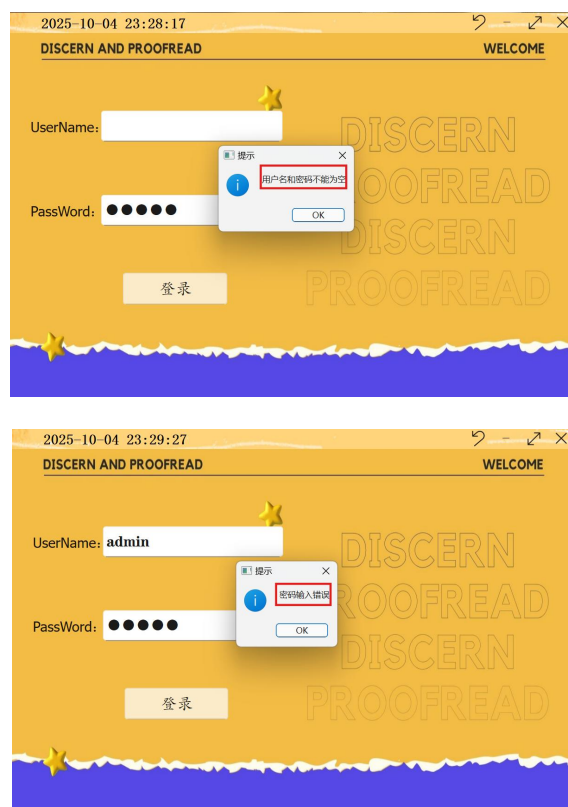


图 5.4 用户（密码）为空与登录密码错误

5.3.2 图像采集与传输测试

硬件端摄像头采集图像清晰稳定，图像传输过程可靠，图像完整到达上位机。测试过程中无丢帧或延迟超过 500ms 情况，确保实时性要求满足，并为后续表盘检测提供高质量输入数据。图像上传测试结果如图 5.5 所示，图像拍摄测试结果如图 5.6 所示。

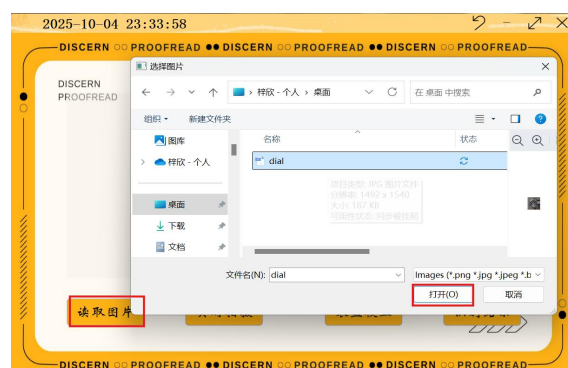




图 5.5 图像上传

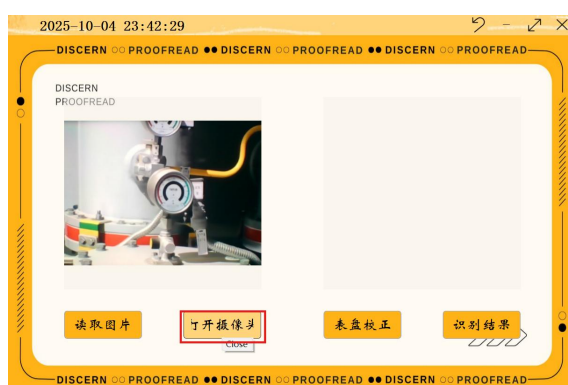


图 5.5 图像拍摄

5.3.3 表盘检测与读数计算测试

YOLOv8 模型检测准确率高，表盘和指针均能正确识别。几何计算得出的读数平均误差小于 1%，在不同光照和角度条件下均保持稳定输出，满足工业巡检对精度和实时性的要求。表盘检测与分割测试结果如图 5.6 所示，输入非法图片与指针检测测试结果如图 5.7 所示，最终读数获取测试结果如图 5.8 所示。



图 5.6 表盘检测与分割



图 5.7 输入非法图片与指针检测



图 5.8 最终读数获取

5.3.4 图像校正与增强测试

对倾斜、模糊或复杂背景的表盘图像进行校正和增强处理，输出结果清晰且带透明背景。经过增强后的图像对后续识别模块提供了可靠输入，保证了系统整体识别流程的准确性和稳定性。图像校正与增强测试结果如图 5.9 所示。



图 5.9 图像校正与增强

5.3.5 数据存储与回溯测试

识别任务、图片记录及框信息均能正确写入数据库。历史任务可查询，数据完整性和一致性良好。系统能按用户和任务进行追踪管理，支持长期数据存储与分析，满足实际应用需求。数据库设计如图 5.10 所示，下属各表存储测试结果如图 5.11 和 5.12 所示所示。

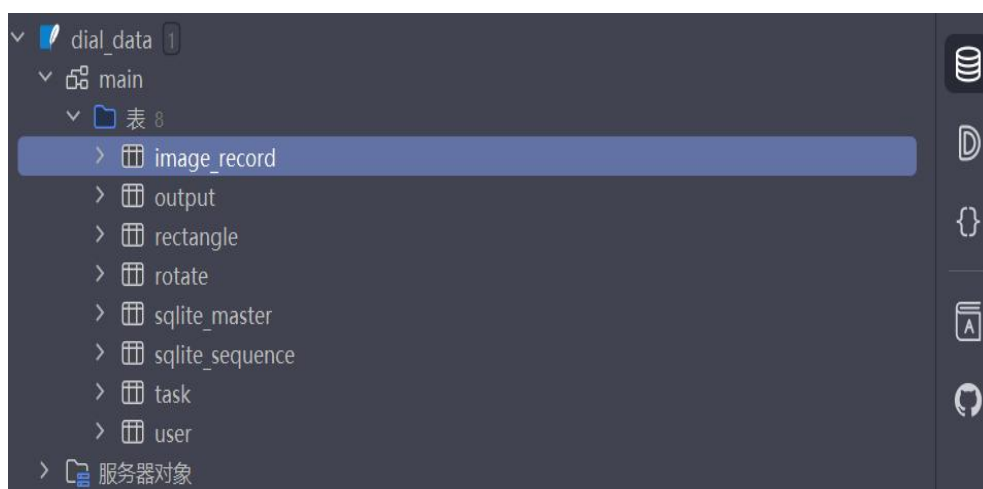


图 5.10 数据库设计

output (1 rows)

SELECT * FROM 'output' LIMIT 0,30

执行

id	task_id	start_cx	start_cy	end_cx	end_cy	mid_cx	mid_cy	pointer_x	pointer_y	center_x	center_y	angle1	ang
1	3	98.282127	292.152832	299.785156	293.38446	202.325134	42.091278	278.654327	77.625031	199.676060	16078457	187.664421	31995018 272.4226 171.

image_record (3 rows)

SELECT * FROM 'image_record' LIMIT 0,30

执行

image_id	task_id	stage	image_path	created_at
1	1	yolo检测	database\dial_detect.jpg	2025-10-04 18:35:36
2	2	原图	D:/OneDrive/Desktop/dial.jpg	2025-10-04 18:35:36
3	2	校正增强	database/dial_perspective_corrected.png	2025-10-04 18:35:49

rectangle (1 rows)

SELECT * FROM 'rectangle' LIMIT 0,30

执行

id	task_id	classes_id	cx	cy	x1	y1	x2	y2	x3	y3	x4	y4
1	1	0	705	835.5	242	1297	1168	1297	1168	374	242	374

图 5.11 下属各表存储 (1)

rotate (5 rows)

SELECT * FROM 'rotate' LIMIT 0,30

执行

id	task_id	classes_id	cx	cy	x1	y1	x2	y2	x3
1	3	0	98.2821273803711	292.15283203125	96.00285339355469	298.7547607421875	104.85693359375	289.79644775390625	100.56141
2	3	1	202.32513427734375	42.091278076171875	198.95188903808594	48.683475494384766	205.85301208496094	48.6020393371582	205.6983
3	3	2	278.6543273925781	77.62503051757812	277.9733581542969	83.73421478271484	284.1756286621094	74.92302703857422	279.3352
4	3	2	283.81341552734375	70.64610290527344	282.84423828125	77.19268798828125	289.72552490234375	67.67225646972656	284.7825
5	3	0	299.78515625	293.38446044921875	302.31219482421875	300.63140869140625	307.13690185546875	295.588134765625	297.2581

sqlite_sequence (6 rows)

SELECT * FROM 'sqlite_sequence' LIMIT 0,30

执行

name	seq
user	1
task	3
rectangle	1
image_record	3
rotate	5
output	1

task (3 rows)

SELECT * FROM 'task' LIMIT 0,30

执行

task_id	user_id	timestamp	result_value
1	1	2025-10-04 18:35:35	
2	1	2025-10-04 18:35:36	0.5296
3	1	2025-10-04 18:35:51	

user (1 rows)

SELECT * FROM 'user' LIMIT 0,30

执行

user_id	username	password	created_at
1	admin	8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca1202...	2025-10-04 18:35:28

图 5.12 下属各表存储 (2)

参考文献

- [1] 桂劲松等.物联网系统设计（第2版）[M].北京：电子工业出版社,2017.
- [2]王嘉乐,宋晓茹,邱豆豆,等. 基于深度学习的低照度环境下图像拼接算法[J/OL].
激光杂志,1-9[2025-09-22].<https://link.cnki.net/urlid/50.1085.TN.20250918.1500.002>.
- [3]郭峰,李京,刘宗鹭. AI 技术在自动化仪表智能识别系统中的应用[J].信息记录材料,2025,26(09):80-82.DOI:10.16009/j.cnki.cn13-1295/tq.2025.09.068.
- [4]王斌虎,刘文,刘岩,等. 一种基于深度学习的指针式仪表读数识别方法[J/OL].自
动化技术与应用,1-6[2025-09-22].<https://link.cnki.net/urlid/23.1474.TP.20250821.1426.006>.
- [5]赵超迪,王新春,杨培宏. 改进 RT-DETR 的指针式仪表读数识别[J/OL].激光与光
电子学进展,1-23[2025-09-22].<https://link.cnki.net/urlid/31.1690.TN.20250815.1535.100>.
- [6]杨海霞.基于深度学习的变电站仪表图像读数识别研究[D].安徽理工大学,2025.
- [7]张涛. 牵引变电所指针式仪表的自动识别研究[J].电气化铁道,2025,36(03):53-56
+65.DOI:10.19587/j.cnki.1007-936x.2025.03.012.
- [8]苏振威. 面向危险环境的多算法集成化工仪表读数识别系统设计[J].自动化应
用,2025,66(09):207-209+220.DOI:10.19769/j.zdhy.2025.09.060.
- [9]赵高长,张磊,武风波. 改进的中值滤波算法在图像去噪中的应用[J].应用光学,20
11,32(04):678-682.
- [10]林开颜,吴军辉,徐立鸿. 彩色图像分割方法综述[J].中国图象图形学报,2005,(0
1):1-10.
- [11]王树文,闫成新,张天序,等. 数学形态学在图像处理中的应用[J].计算机工程与
应用,2004,(32):89-92.

附录 关键部分程序清单

```
# 仪表指针读数计算核心算法

# final.py

# -*- coding: utf-8 -*-

import os

import re

import numpy as np

from datetime import datetime

def read_scale_latest_file(file_path):

    """读取检测结果文件，提取各标注框的中心坐标与角点信息。"""

    with open(file_path, 'r', encoding='utf-8') as f:

        content = f.read()

    pattern = r"Label: (.+?), Num: (\d+)\s+Center: \((.+?), (.+)\)\s+Corner\n\nCorner_Lower_Left \((x1,y1)\):\((.+?), (.+)\)\n\nCorner_Lower_Right \((x2,y2)\):\((.+?), (.+)\)\n\nCorner_Upper_Right \((x3,y3)\):\((.+?), (.+)\)\n\nCorner_Upper_Left \((x4,y4)\):\((.+?), (.+)\)"

    matches = re.findall(pattern, content)

    boxes = []

    for m in matches:

        label, num = m[0], int(m[1])

        cx, cy = float(m[2]), float(m[3])

        corners = [(float(m[i]), float(m[i + 1])) for i in range(4, 12, 2)]

        boxes.append({'label': label, 'label_num': num, 'center': (cx, cy), 'corners': corners})

    return boxes
```

```
def find_box_by_label(boxes, label_num):
    """按编号查找指定标注框。"""
    for box in boxes:
        if box['label_num'] == label_num:
            return box
    return None

def perpendicular_bisector(p1, p2):
    """求两点连线的垂直平分线。"""
    mid_x, mid_y = (p1[0] + p2[0]) / 2, (p1[1] + p2[1]) / 2
    if p1[0] == p2[0]:
        return (mid_x, 0), (mid_x, 1)
    elif p1[1] == p2[1]:
        return (0, mid_y), (1, mid_y)
    else:
        k = (p2[1] - p1[1]) / (p2[0] - p1[0])
        k_perp = -1 / k
        return (mid_x - 1, mid_y - k_perp), (mid_x + 1, mid_y + k_perp)

def intersection(p1, p2, p3, p4):
    """计算两条直线交点。"""
    A1, B1 = p2[1] - p1[1], p1[0] - p2[0]
    C1 = A1 * p1[0] + B1 * p1[1]
    A2, B2 = p4[1] - p3[1], p3[0] - p4[0]
    C2 = A2 * p3[0] + B2 * p3[1]
    det = A1 * B2 - A2 * B1
    if det == 0:
        raise ValueError("线段垂直平分线无交点")
```

```
x = (C1 * B2 - C2 * B1) / det
```

```
y = (A1 * C2 - A2 * C1) / det
```

```
return x, y
```

```
def angle_between(v1, v2):
```

```
    """计算两向量夹角（角度制）。"""
```

```
    u1, u2 = v1 / np.linalg.norm(v1), v2 / np.linalg.norm(v2)
```

```
    angle = np.arccos(np.clip(np.dot(u1, u2), -1.0, 1.0))
```

```
    return np.degrees(angle)
```

```
def compute_gauge(boxes, log_file=None):
```

```
    """根据刻度与指针位置计算仪表读数。"""
```

```
    min_box = min(boxes, key=lambda x: x['center'][0])
```

```
    max_box = max(boxes, key=lambda x: x['center'][0])
```

```
    scale2_box = find_box_by_label(boxes, 1)
```

```
    pointer_box = find_box_by_label(boxes, 2)
```

```
    if not scale2_box or not pointer_box:
```

```
        raise ValueError("缺少刻度或指针信息")
```

```
    (x_sf, y_sf) = min_box['center']
```

```
    (x_ef, y_ef) = max_box['center']
```

```
    (x_zf, y_zf) = scale2_box['center']
```

```
    (x_pf, y_pf) = pointer_box['center']
```

```
    l1 = perpendicular_bisector((x_sf, y_sf), (x_zf, y_zf))
```

```
    l2 = perpendicular_bisector((x_zf, y_zf), (x_ef, y_ef))
```

```
    l3 = perpendicular_bisector((x_sf, y_sf), (x_ef, y_ef))
```

```

c_x, c_y = intersection(*l1, *l2)
c_x2, c_y2 = intersection(*l2, *l3)
c_x3, c_y3 = intersection(*l1, *l3)
c_x, c_y = (c_x + c_x2 + c_x3) / 3, (c_y + c_y2 + c_y3) / 3
if abs(c_x - c_y) > 100: c_y = c_x

vec_rs = np.array([x_sf - c_x, y_sf - c_y])
vec_rp = np.array([x_pf - c_x, y_pf - c_y])
vec_re = np.array([x_ef - c_x, y_ef - c_y])
a1 = angle_between(vec_rs, vec_re)
a2 = angle_between(vec_rs, vec_rp)
if a1 < 180: a1 = 360 - a1

ans = 1.0000 * a2 / a1 - 0.1000

timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
log = (f'==== 时间: {timestamp} ==== \n'
      f'起点中心: ({x_sf},{y_sf}) \n'
      f'终点中心: ({x_ef},{y_ef}) \n'
      f'中点中心: ({x_zf},{y_zf}) \n'
      f'指针端点: ({x_pf},{y_pf}) \n'
      f'圆心坐标: ({c_x:.2f},{c_y:.2f}) \n'
      f'夹角 a1: {a1:.4f} ° \n'
      f'夹角 a2: {a2:.4f} ° \n'
      f'仪表读数: {ans:.4f} \n'
      "----- \n")

print(log)

if log_file:

```



```

os.makedirs(os.path.dirname(log_file), exist_ok=True)

with open(log_file, "a", encoding="utf-8") as f:

    f.write(log)

return ans

```

仪表图像增强与校正核心代码

dial_calibration.py

-*- coding: utf-8 -*-

import cv2

import numpy as np

import os

def enhance_image(img, alpha=1.2, beta=10, gamma=1.0, clahe_clip=2.0, clahe_grid=8):

"""图像增强：亮度、对比度、Gamma 校正与 CLAHE 自适应均衡。"""

adjusted = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)

inv_gamma = 1.0 / gamma

table = np.array([(i / 255.0) ** inv_gamma * 255 for i in np.arange(256)]).astype("uint8")

gamma_corrected = cv2.LUT(adjusted, table)

lab = cv2.cvtColor(gamma_corrected, cv2.COLOR_BGR2LAB)

l, a, b = cv2.split(lab)

clahe = cv2.createCLAHE(clipLimit=clahe_clip, tileGridSize=(clahe_grid, clahe_grid))

l = clahe.apply(l)

enhanced = cv2.merge((l, a, b))

enhanced = cv2.cvtColor(enhanced, cv2.COLOR_LAB2BGR)

return enhanced

```

def dial_calibration(input_path="database/new.jpg",
                    output_path="database/dial_perspective_corrected.png",
                    size=400):
    """自动检测表盘、裁剪并增强，输出带透明背景的圆形 PNG 图像。"""
    if not os.path.exists(input_path):
        print(f"✘ 输入图片不存在: {input_path}")
        return False

    img = cv2.imread(input_path)
    if img is None:
        print(f"✘ 无法读取图片: {input_path}")
        return False

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = cv2.medianBlur(gray, 5)
    circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1.2, minDist=100,
                               param1=100, param2=30, minRadius=100, maxRadius=0)

    if circles is not None:
        x, y, r = np.uint16(np.around(circles))[0][0]
        r = int(r * 1.08)
        crop = img[y - r:y + r, x - r:x + r]
        corrected = cv2.resize(crop, (size, size))
        enhanced = enhance_image(corrected, alpha=1.3, beta=15, gamma=1.1)

        result = np.zeros((size, size, 4), dtype="uint8")

```

```

mask = np.zeros((size, size), dtype="uint8")

cv2.circle(mask, (size // 2, size // 2), size // 2 - 2, 255, -1)

result[:, :, :3] = enhanced

result[:, :, 3] = mask

cv2.imwrite(output_path, result)

print(f"☑ 已保存增强后的图片: {output_path}")

return True

else:

    print("✗ 未检测到表盘圆，请调整 HoughCircles 参数")

    return False

if __name__ == "__main__":

    dial_calibration("database/new.jpg", "database/dial_perspective_corrected.png")

```